

Memory Efficient Doubly Linked List
–insertAfterElement and Time Complexity

Program:

```
void insertAfterElement(int data, int element)
{
    Node *curr = head;
    Node *prev = nullptr;
    Node *next;

    // Search for element
    while (curr != nullptr && curr->data != element)
    {
        next = XOR(prev, curr->npx);
        prev = curr;
        curr = next;
    }

    if (curr == nullptr)
    {
        cout << "Element " << element << " not found in the
list." << endl;
        return;
    }

    // Element found at 'curr'. Insert 'newNode' between
'curr' and 'next'.
    next = XOR(prev, curr->npx); // Decode the node after
curr

    Node *newNode = (Node *)malloc(sizeof(Node));
    newNode->data = data;

    // Update pointers using XOR logic
    newNode->npx = XOR(curr, next);
    curr->npx = XOR(prev, newNode);
```

```

    if (next != nullptr)
    {
        Node *nextNext = XOR(curr, next->npx);
        next->npx = XOR(newNode, nextNext);
    }

    cout << data << " inserted after " << element << "." <<
endl;
}
... .

    case 4:
        cout << "Enter data to insert: ";
        cin >> data;
        cout << "Enter the element to insert after: ";
        cin >> element;
        insertAfterElement(data, element);
        break;

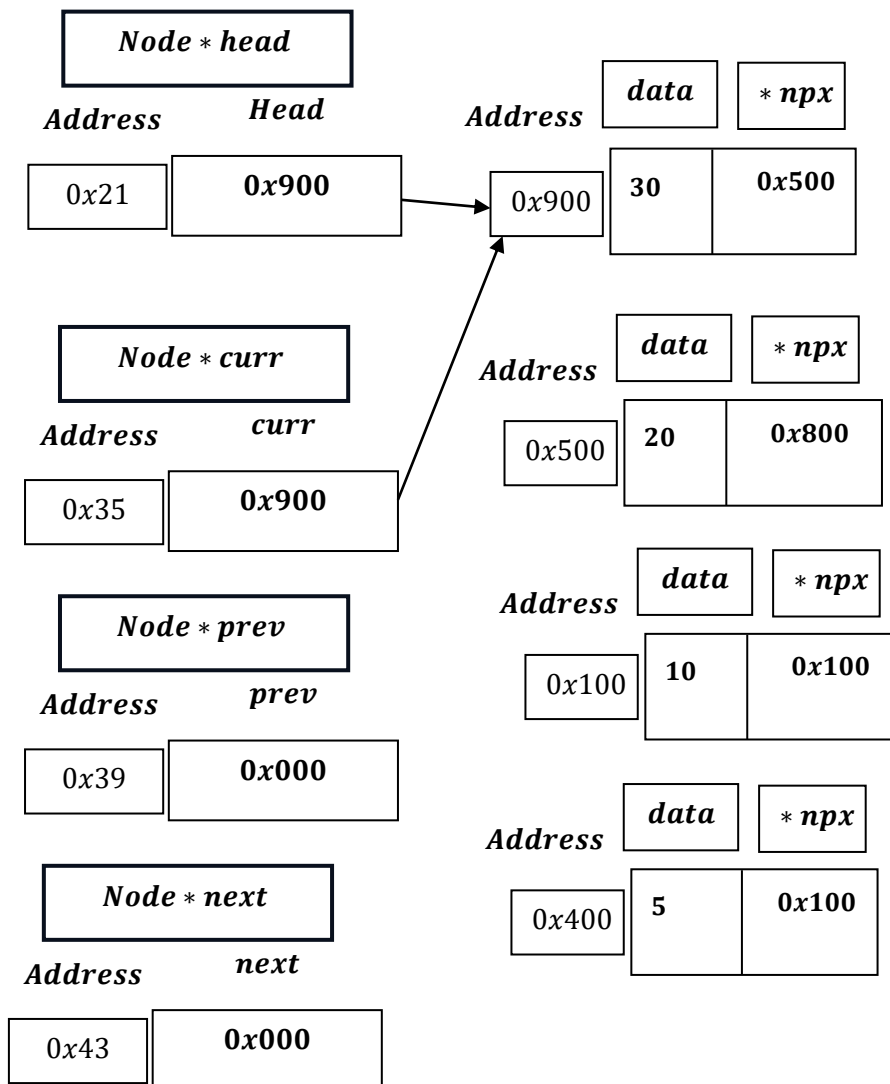
```

Program Analysis:

```

Node *curr = head;
Node *prev = nullptr;
Node *next;

```



```
while (curr != nullptr && curr->data != element)
{
    next = XOR(prev, curr->npx);
    prev = curr;
    curr = next;
}
```

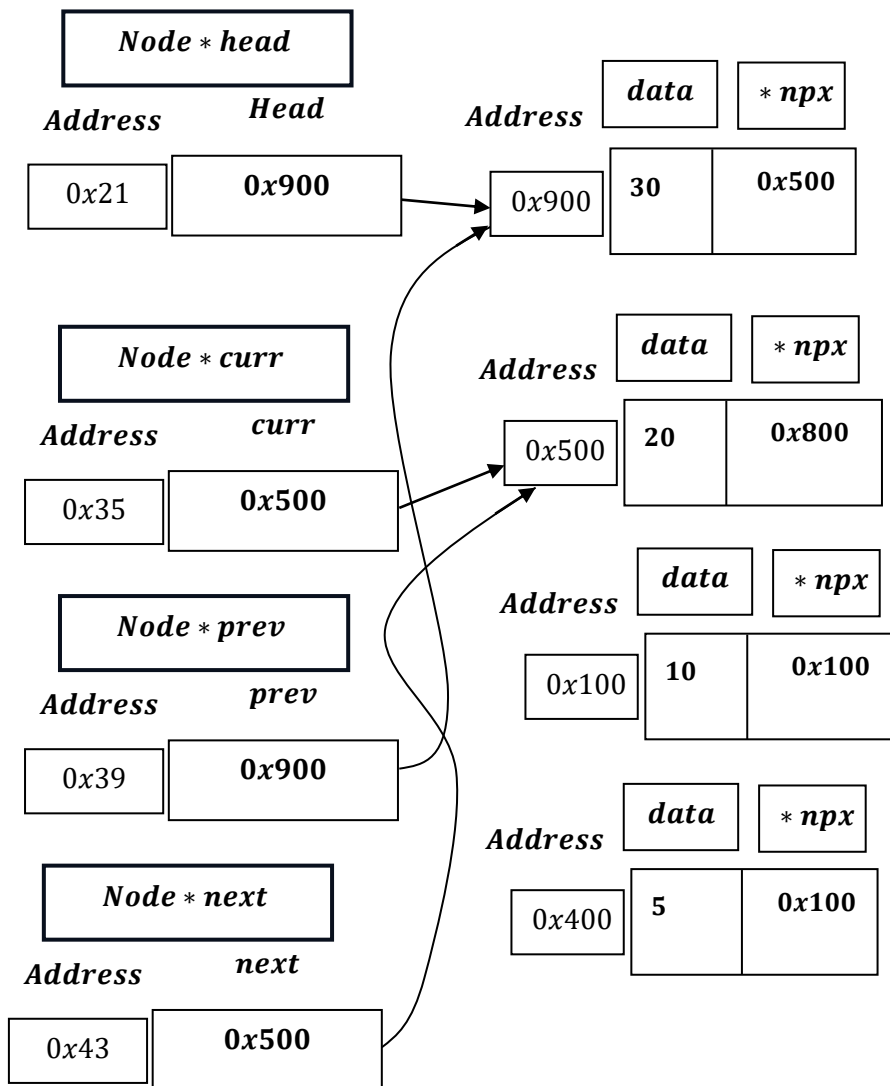
Lets say ,Element = 5 and data = 15;

curr: 0x900 $\neq$ nullptr: 0x000 AND curr $\rightarrow$ data = 30 $\neq$ element = 5 :

next = XOR(prev: 0x000, curr $\rightarrow$ npx: 0x500) = 0x000 $\oplus$ 0x500 = 0x500.

prev = curr = 0x900

curr = next = 0x500

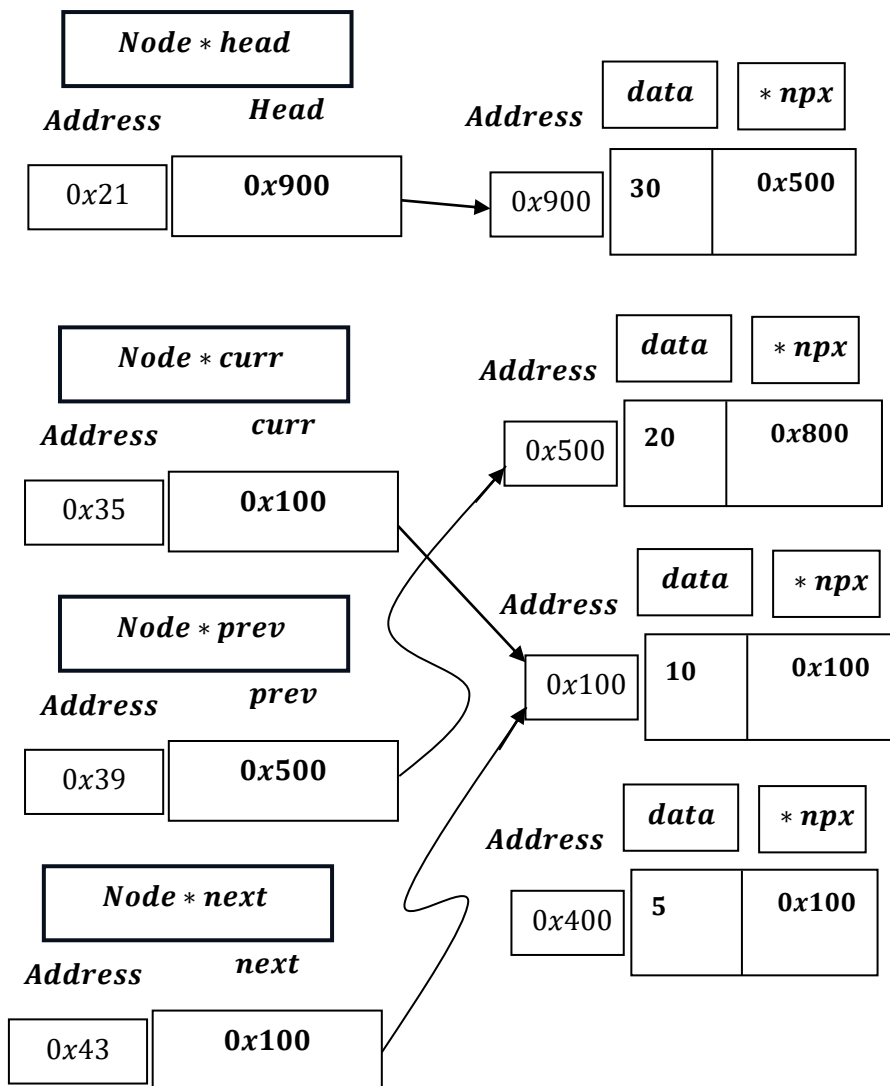


curr: 0x500 $\neq$ nullptr: 0x000 AND curr $\rightarrow$ data = 20 $\neq$ element = 5 :

next = XOR(prev: 0x900, curr $\rightarrow$ npx: 0x800) = 1001₂ $\oplus$ 1000₂ = 0001₂ = 0x100

prev = curr = 0x500

curr = next = 0x100

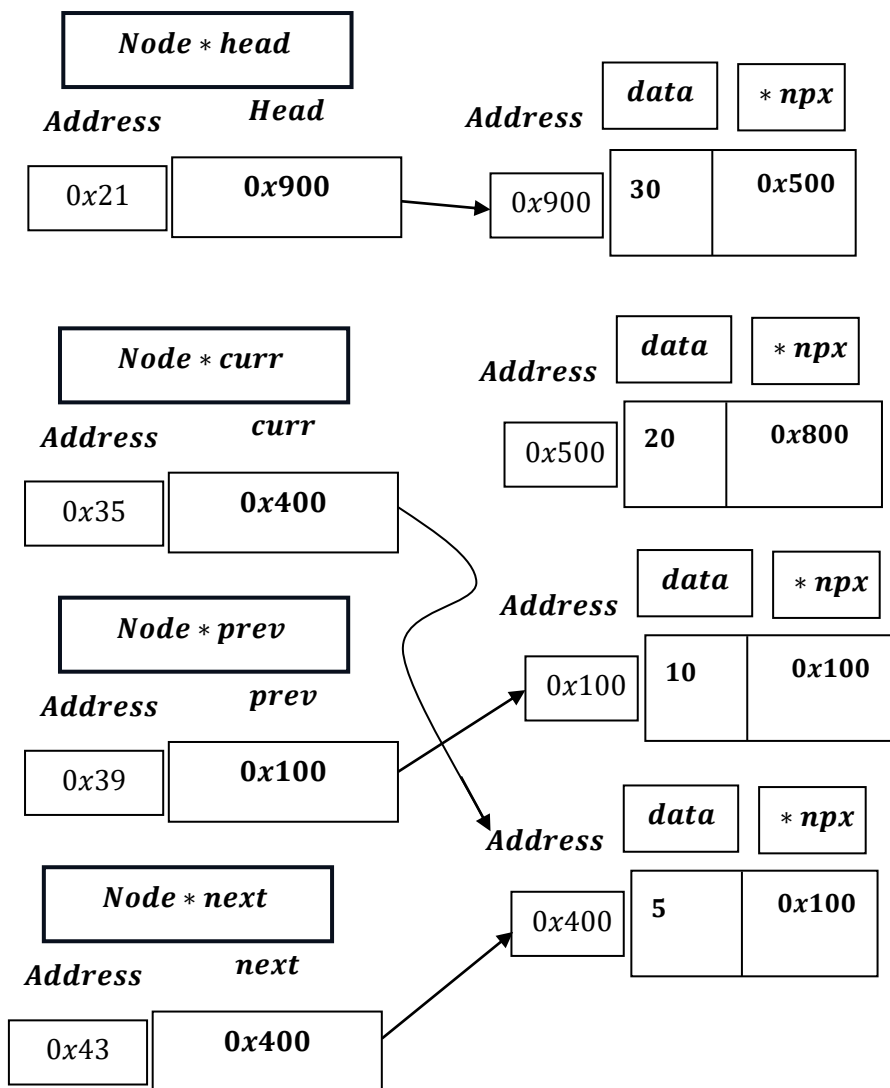


$curr: 0x100 \neq nullptr: 0x000$ AND $curr \rightarrow data = 10 \neq element = 5$:

$next = XOR(prev: 0x500, curr \rightarrow npx: 0x100) = 0101_2 \oplus 0001_2 = 0100_2 = 0x400$

$prev = curr = 0x100$

$curr = next = 0x400$

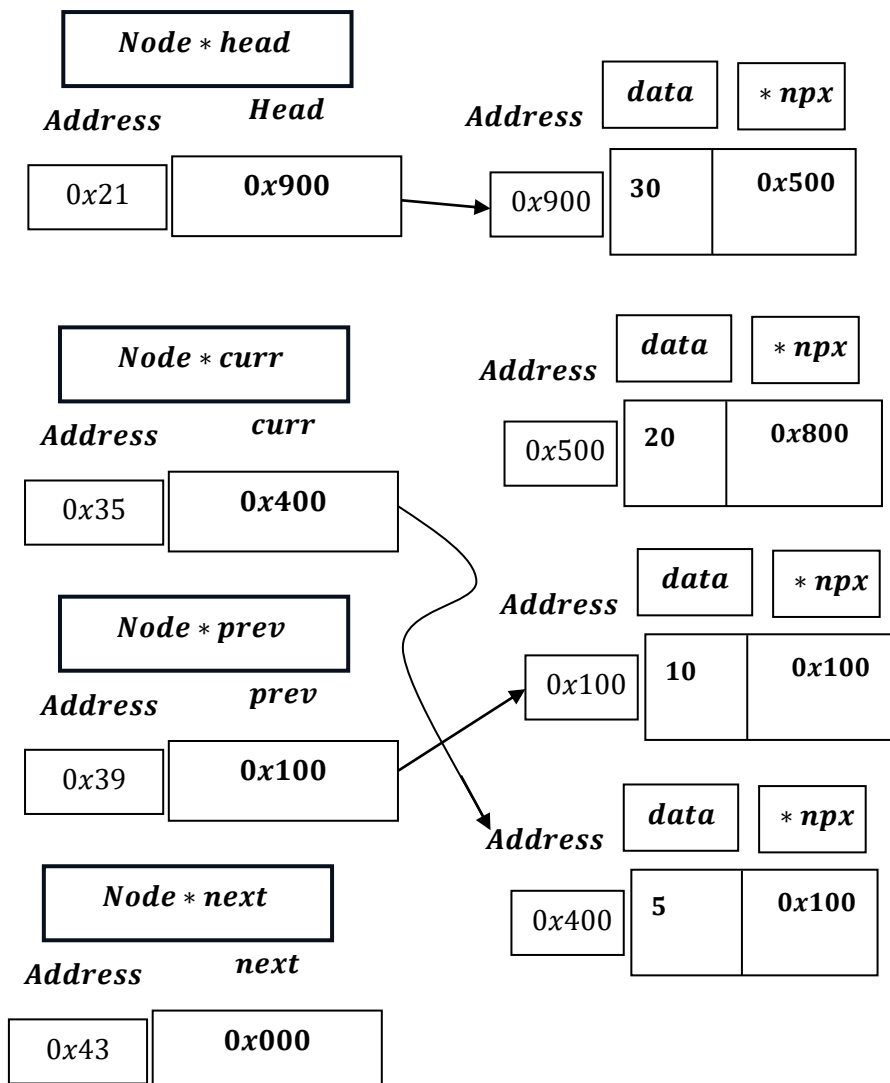


*curr: 0x400 $\neq$ nullptr: 0x000 [True] AND curr $\rightarrow$ data = 5 $\neq$ element = 5 [False]
 $\rightarrow$ is False , hence loop exits.*

```
next = XOR(prev, curr->npx);
```

next = XOR(prev: 0x100 $\oplus$ curr $\rightarrow$ npx: 0x100);

0001₂ $\oplus$ 0001₂ = 0000₂ = 0x000.



```
Node *newNode = (Node *)malloc(sizeof(Node));
```

→ The size of *newNode* will be equal to the size of a *Node* structure and *newNode* is declared as a pointer to *Node* structure.

→ The size of a pointer (*newNode*) is typically 8 bytes on most modern 64 – bit systems, regardless of the size of the structure it points to.

→ *The size of the Node structure itself is determined by the sum of the sizes of its members:*

→ *data(an integer) is typically 4 bytes.*

→ *npx(a pointer to another Node) is typically 8 bytes.*

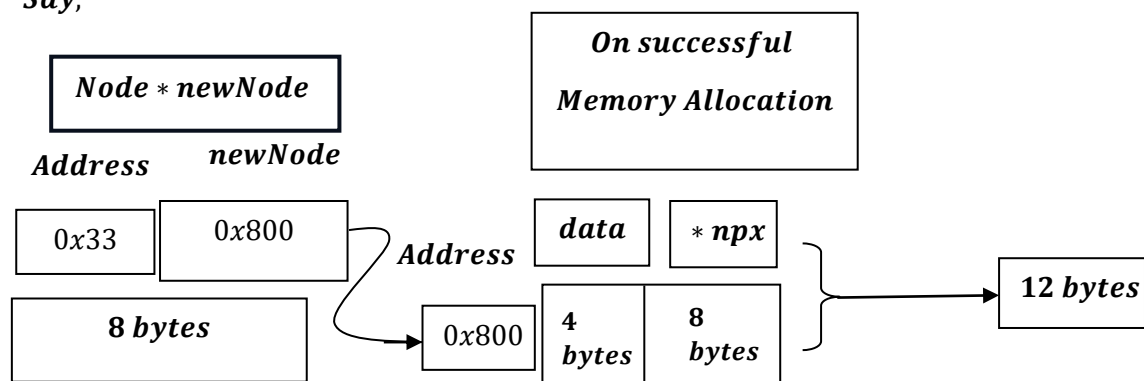
→ *Therefore ,the size of the Node structure is typically 12 bytes(4 bytes for data + 8 bytes for npv) .*

```
Node *newNode = (Node *)malloc(sizeof(Node));
```

What does it mean?

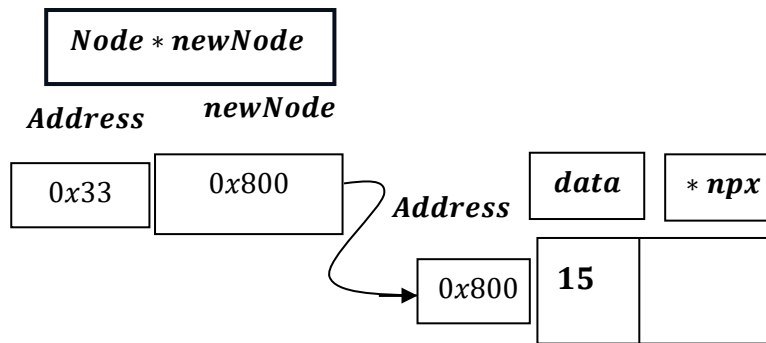
→ *newNode will try to allocate memory for data i.e. 4 bytes ,npv pointer variable i.e. typically 8 bytes,i.e. total 12 bytes,as discussed earlier ,the size of a pointer (newNode) is typically 8 bytes on most modern 64 – bit systems, regardless of the size of the structure it points to.*

Say,



```
newNode->data = data;
```

Say, data = 15. Then ,newNode → data = data = 15.

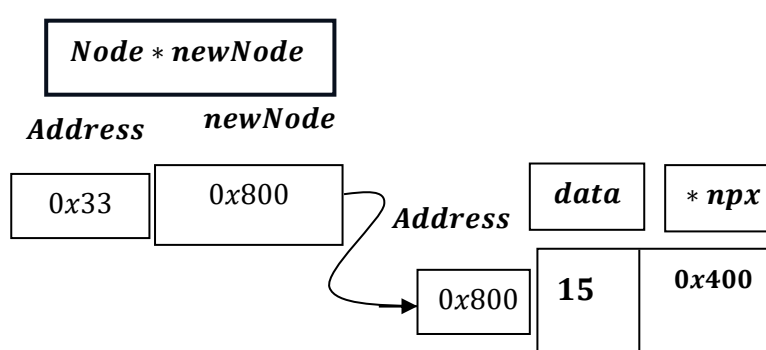


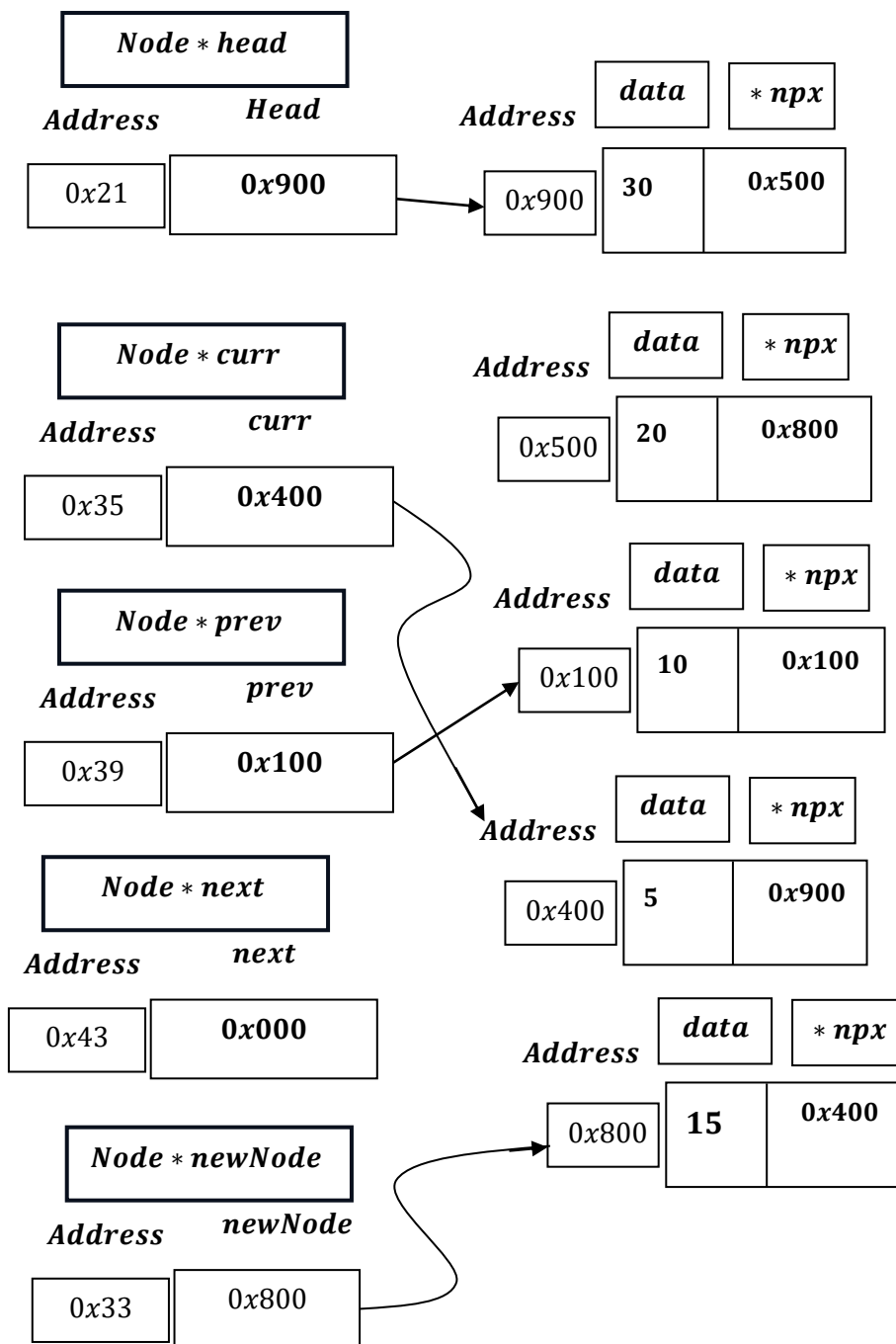
```
newNode->npx = XOR(curr, next);
curr->npx = XOR(prev, newNode);
```

$newNode \rightarrow npx = XOR(curr: 0x400, next: 0x000) = 0x400.$

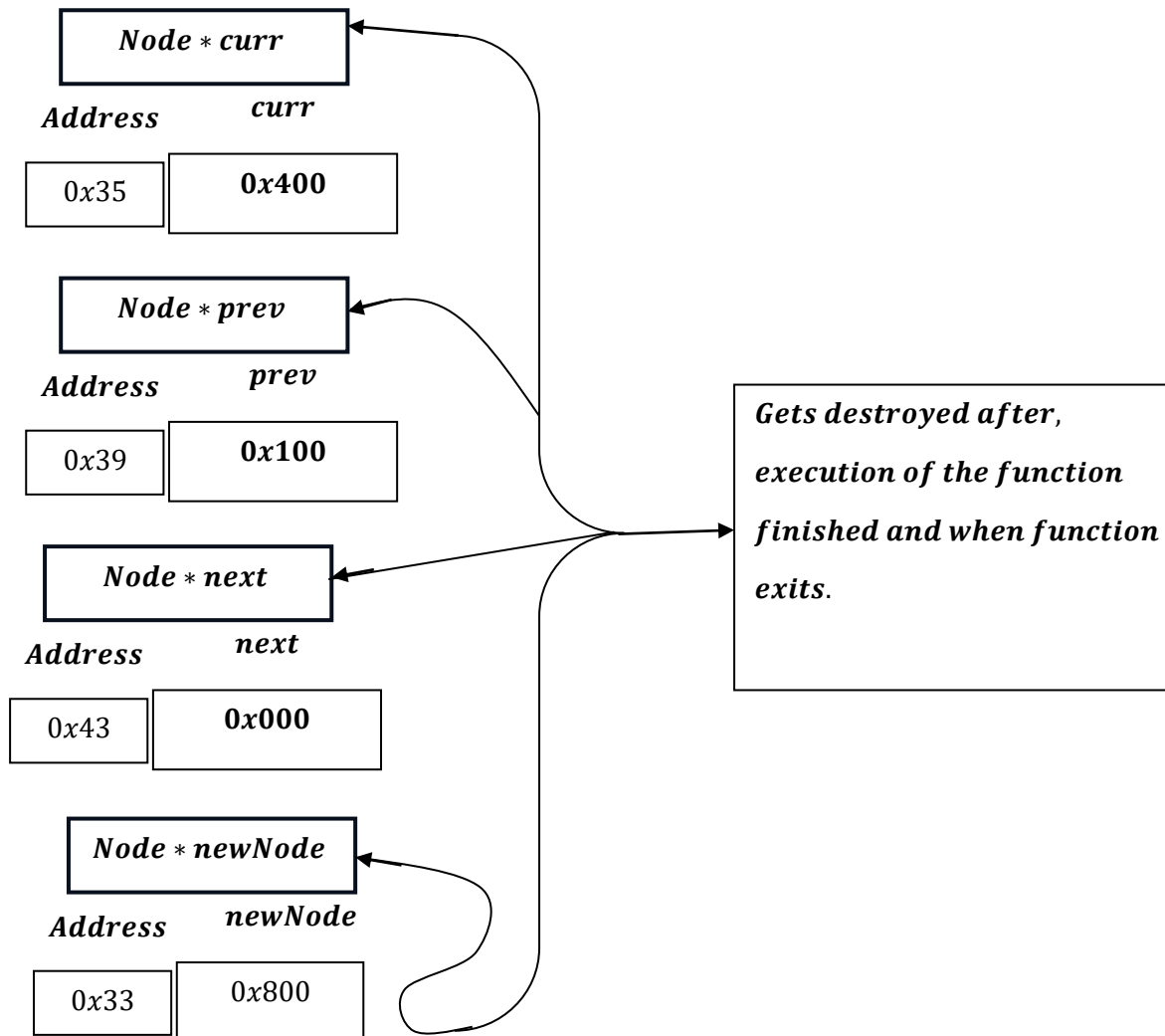
$curr \rightarrow npx = XOR(prev, newNode) = (prev: 0x100, newNode: 0x800) =$

$0001_2 \oplus 1000_2 = 1001_2 = 0x900.$





As , functions ends:



And we are left with:

